# Fixed Charts, Exact Cores, and Rational Moments

A differential review of Asymptotic

10 September 2026

| | |
|---|---|
| Repository | `VladimirReshetnikov/Asymptotic` |
| Reviewed commit | `8cee870994f506b501bae3ea6bd4a3a7edb895c1` |
| Main result | One high-priority source-level remainder defect, with an elementary counterexample at every finite depth. |
| Companion work | A guarded candidate patch, two additive development prototypes, and reproducibility artifacts. |

**Evidence at a glance**

**32 independent Python test methods passed.** These are mathematical models, rational-algorithm checks, numerical samples, and patch-fragment fixtures—not repository tests. The public Wolfram/Mathics witnesses and eight desired-contract tests are **unexecuted**. Source-predicted behavior is identified explicitly throughout.

The review is screened against the maintained prior-review registers, the five-wave index, and targeted original material. Previously recorded findings and general roadmap proposals are not presented again as new work.

## Executive assessment

The most consequential incremental issue found is not an incorrect coefficient formula. It is a missing restriction on a coordinate option. The exponential-core inverse constructor allows `SourceShift` to depend on the target variable, although its remainder theorem assumes a fixed source chart. For the source function $e^x + 1$, the choice $\sigma(y) = -|y|$ leaves the displayed inverse coefficients correct but changes the reported error scale from an algebraic scale into an exponentially smaller one. On the positive tail, the ratio of the actual error to that scale diverges at least as $e^{(N+1)y}/(N+1)$ at every finite retained depth $N$.

The source mechanism is explicit in the constructor and its core parser: the input guard excludes the source symbol from the shift but not the target symbol; the shifted exponential becomes a core with target-dependent amplitude; the remainder scale then suppresses that amplitude as though it were fixed. The public call is a *source-predicted witness*, not a recorded kernel evaluation. The mathematical contradiction does not depend on a numerical root finder or a difficult special-function identity. Section 3 gives the complete derivation and a minimal structural repair.[1, 2]

Two additional items are development opportunities, not extra correctness findings. First, an explicit regrouping of a constant perturbation into the core's target offset can expose an exact inverse that the selected unshifted core represents by an infinite correction sequence. Second, the already implemented Lerch expansion can use a small exact rational moment engine instead of separate negative-order polylogarithm and general simplification calls for rational parameters. The supplied prototypes preserve this narrow scope; they do not redesign the package's option machinery or its general special-function architecture.

| ID | Classification | Incremental conclusion |
|---|---|---|
| F1 | High priority | Reject a target-dependent source translation before constructing an exponential-core inverse. Otherwise the fixed-chart error theorem can yield a false remainder. |
| E1 | Opportunity | Offer opt-in absorption of a fixed constant perturbation into the core offset. Preserve a user's deliberate choice of an unabsorbed core. |
| E2 | Opportunity | Specialize rational Lerch moments with an integer numerator recurrence and rational arithmetic; retain the existing domain and tail theorem. |

The review does not establish full compatibility with either kernel. Wolfram execution-service calls failed, a local Mathics installation was unavailable, and no complete local checkout was obtained. Commit-pinned source retrieval was successful. The independent Python suite passed 32 methods with no failures, errors, or skips; its population is deliberately separate from the repository's historical acceptance records. The supplied WL code received only a limited delimiter, string, and comment check.

# Contents

# 1 Scope, method, and the exclusion boundary

## 1.1 What the review actually covers

This is a differential review of the pinned snapshot, not a restatement of the project's accumulated backlog. The canonical implementation is modular under `src/Kernel`; the repository-root standalone distribution is generated from that source. The kernel entry point coordinates the public declarations, coefficient algebra, constructors, operations, and conditional Mathics adapters. The source map and existing build procedure are documented by the repository itself.[4]

The inspection followed several paths across those modules: ordinary and specialized inverse construction; source and target coordinate transformations; Gamma and Barnes forward and inverse normalization; finite logarithmic hierarchies; Fourier coefficients; native special-function admission and import; composite error arithmetic; selected interval helpers; and Mathics-specific assumptions, algebra, Taylor, calculus, call, and core-function adapters. This breadth was useful for comparing how parameter scope and error information cross module boundaries. It was not a complete line-by-line review of every test, document, generated file, or implementation branch.

The accompanying `evidence/source-map.json` records files and inspected ranges. In particular, large modules such as `AsymptoticAnalysis.wl` and `SeriesOperations.wl` were read in selected ranges rather than certified in their entirety. A truncated source response is not counted as proof that its unseen suffix was inspected. Search endpoints returned incomplete empty results, so they were not used as evidence that a name or problem was absent.

## 1.2 Prior work was an exclusion set, not a source of fresh findings

The maintained register includes crosswalks for earlier reports and a broad mathematical and architectural roadmap. The wave-3 and wave-4 intakes extend that register. The wave-5 index is especially important because it describes additional, not-yet-consolidated findings and explicitly identifies retired duplicates. Omitting it would incorrectly rediscover several recent issues.[5, 6, 7, 8]

The exclusion process compared candidate mechanisms with those maintained records and, when the closest neighbor mattered, with original report material. This is not a claim to have compared every paragraph of every earlier PDF. It is a claim that the three items retained here have a concrete, stated distinction from the closest recorded obligations.

For F1, the closest neighbors concern parameter capture during composition, retained source symbols in a user-supplied inverse, and the general lack of uniform parameter asymptotics. The new ingress is different: the target is introduced into a supposedly fixed chart *during the initial constructor call*. No composition, saved result, or user-supplied inverse is needed. Report 43's source-symbol finding was checked directly to avoid reversing that distinction.[5, 9]

For E1, the distinction is from stopping a recurrence after exact termination. Here the coefficients about the chosen core never terminate; changing the explicitly chosen core makes the solution exact. For E2, the distinction is a concrete rational arithmetic kernel for an existing adapter, not another recommendation to add caches, a general coefficient registry, or a larger list of special functions. Those broad proposals are not repeated here.

### 1.3 Four evidence categories

| Category | Meaning in this report |
|---|---|
| Source observation | A condition, formula, or control-flow edge visible at the pinned commit. |
| Mathematical result | A deduction proved from explicit expressions and hypotheses in the article. |
| Independent execution | Python calculations on the stated model or companion algorithm. They do not run the package. |
| Kernel prediction | Expected public Wolfram or Mathics behavior from source inspection; pending actual evaluation. |

This distinction prevents two common inference errors. A correct independently tested recurrence does not show that the package successfully reaches that recurrence. Conversely, a service failure does not invalidate an elementary counterexample to the formula visible in source. Both facts belong in the result, without promoting one evidence category into another.

## 2 Implementation context and cross-runtime assessment

### 2.1 The contract boundary that matters here

The package's analytic results are not merely lists of coefficients. The inspected constructors retain a coordinate, a finite expression, a remainder scale, domain information, and source provenance. Native delegation is a separate path with its own result and order conventions; the analytic special-function importer is not the same operation as merely wrapping an arbitrary native result. These distinctions are documented in the kernel map and visible in the specialized importer.[4, 13]

F1 crosses one of those analytic boundaries. The displayed coefficient expression can simplify to the right answer while its stored error contract describes a different asymptotic scale. A check of coefficients alone, or substitution into an equation at a few points without comparing the stated error, can therefore miss the problem. The relevant object is the pair

$$\big(A_N(y), R_N(y)\big), \qquad x_*(y) - A_N(y) = O(R_N(y)),$$

not $A_N$ in isolation. The constant and threshold implicit in this statement must be independent of $y$.

The inspected exponential-core constructor already has an appropriate analytic idea: retain a recognized exact core, construct complete perturbative sectors, and attach a tail for the entire omitted model rather than treating the first omitted coefficient as the whole error. Its separate transport of a declared input remainder is also explicit. F1 does not dispute that design; it identifies an option value for which the fixed-data hypothesis used by the design is not enforced.[1]

### 2.2 Why Wolfram and Mathics need separate observations

Mathics support in the repository is not implemented by assuming that all built-ins behave identically. The entry point conditionally loads helpers in a private adapter context, and some package-owned construction sites receive late overrides. The Mathics files inspected here include assumption reasoning, exact algebra, defining Taylor series, finite sums, held extraction, and selected special-function cores. The official Wolfram path does not load these adapters as global replacements for `System` functions.[4, 10, 11, 12]

This makes the F1 witness a useful example of why a shared source defect and an identical public symptom are different claims. The native meaning of `Abs[y]` supplies a real nonnegative expression without an assumption that $y$ itself is real.[21] The inspected Mathics realness helper has a more

restrictive unary rule for `Abs`: it proves the argument real first. A fallback simplifier may affect the actual outcome. Thus the present Mathics constructor may already refuse the witness for an earlier proof limitation. That is not established either way by this review.[10]

The proposed fix does not rely on either simplifier. It rejects structural dependence on the target before the coordinate construction. This creates the same intended contract in both runtimes while retaining the legitimate use of the target in `CoreInverse`. Actual execution remains necessary to confirm the public failure tag, option evaluation, and neighboring accepted cases.

### 2.3 Other inspected mathematical paths

The comparison also checked the signs and normalizations of the finite Gamma/Barnes residual equations, the Barnes forward shift reduction, the logarithmic Euler derivative, and the elementary moment-bound argument in the Lerch path. No additional nonduplicate contradiction was established in those inspected formulas. This is deliberately narrower than calling the modules correct: admission, evaluator behavior, precision transport, and uninspected branches can still matter.[14, 15, 16, 17, 3]

Similarly, the composite error-arithmetic source retains separate error summands and imposes relative or absolute smallness conditions for operations such as logarithms or exponentials. That was relevant context for understanding why an exponentially overstated remainder can contaminate later reasoning, but this report does not assert a new executed downstream failure in composite arithmetic.[18]

## 3 F1: a moving source chart creates a false remainder

### 3.1 Location, severity, and minimal witness

**Classification: high-priority source-level correctness defect.**

The affected operation is `AsymptoticExponentialCoreInverse`.

Its source is `ExponentialCorePerturbation.wl`, which contains the coordinate, exact-inverse, coefficient, and constructor helpers. The exponential parser is in `LambertInverse.wl`.[1, 2]

The candidate public call is:

```
AsymptoticExponentialCoreInverse[
  Exp[x], 1, {x, Infinity}, {y, 0},
  "SourceShift" -> -Abs[y]
]
```

The same construction applies with any nonnegative retained depth in place of zero. The selected positive target tail may be taken as $y > 4$. On that tail, the real inverse of the original source is simply

$$x_*(y) = \log(y - 1). \tag{1}$$

**Execution status of the witness**

No kernel output for this call was obtained. The source predicts the formulas below on the positive tail, and the contradiction between those formulas and the exact inverse is proved independently. A native or Mathics refusal, unevaluated expression, or different output must be recorded as an observation rather than replaced by the prediction.

The severity comes from a false asymptotic error assertion on an elementary example, not from a crash or a missing feature. Even when the finite approximation is useful, an arbitrarily smaller claimed error

can invalidate a consumer's precision choice or its decision that an error is negligible. No quantitative interval certificate is claimed by the original constructor, but a qualitative Big-$O$ assertion still has a precise mathematical meaning.

### 3.2 The missing structural condition

The constructor's variable guard checks, in substance,

```
x === y
! FreeQ[{core, perturbation}, y]
! FreeQ[ass, x | y]
! FreeQ[{shift, requested}, x]
```

It does not check that `shift` is independent of `y`. The coordinate helper checks exactness, independence from the source symbol, and provable realness. Its diagnostic calls the shift a parameter, but the corresponding target-independence condition is absent.[1]

The asymmetry between the shift and the requested core inverse is important. A requested inverse *should* be a function of the target; an allegedly fixed translation *should not*. Applying one blanket prohibition to both values would repair F1 by breaking valid requests. The required distinction is between the two option roles, not between expressions that happen to contain a symbol.

For the witness, the shift is exact, contains no $x$, and is mathematically real. The source reconstruction is

$$x = -|y| + v.$$

On $y > 4$, the core becomes

$$e^x = e^{-y}e^v = a(y)e^v, \qquad a(y) = e^{-y}. \tag{2}$$

The parser explicitly moves a part of the exponent independent of its local source variable into the amplitude. It does not require that amplitude to be independent of the target, because the target has entered through the shift after the original input guard.[2]

### 3.3 The exact core inverse still looks harmless

For a core $ae^v$, the constructor uses the elementary inverse

$$v_0 = \log(y/a).$$

With (2),

$$v_0 = y + \log y, \qquad -y + v_0 = \log y. \tag{3}$$

Both the positive local variable and the reconstructed source tend to $+\infty$. The recorded positivity conditions are satisfied on $y > 4$: $y/a > 1$, $v_0 > 1$, and the derivative factor $v_0$ is positive. Consequently this example is not explained away by a local coordinate tending to the wrong endpoint.

The more subtle failure is that the normalized core amplitude is no longer fixed. Its reciprocal grows exponentially with the target. This growth cancels in the displayed coefficient expression but not in the stored remainder scale.

### 3.4 Derivation of every retained coefficient

The exponential-core coefficient helper starts with a rational expression involving the perturbation, the derivative of the source reconstruction, and

$$D(v) = av^{b-1}(b + cpv^p).$$

For the present case the perturbation is 1, the source derivative is 1, and $b = 0$, $c = p = 1$, so $D(v) = a$. The recurrence inside the helper reduces to

$$Q_0 = \frac{1}{a}, \qquad Q_j = -\frac{j}{a} Q_{j-1}.$$

Multiplication by the final factor $(-1)^n/n!$ therefore yields

$$C_n = -\frac{1}{n a^n}, \qquad n \geq 1. \tag{4}$$

This is a direct simplification of the visible recurrence, and the independent Python tests also compare its exact rational specialization with the closed form.[1]

The stored sector variable is

$$q = e^{-v_0} = \frac{a}{y} = \frac{e^{-y}}{y}. \tag{5}$$

Combining (4) and (5) gives

$$C_n q^n = -\frac{1}{n y^n}.$$

Thus the displayed approximation through depth $N$ is the correct finite logarithmic expansion

$$A_N(y) = \log y - \sum_{n=1}^{N} \frac{1}{n y^n}. \tag{6}$$

The bad chart has disappeared from the finite expression. Checking the first several coefficients, comparing them against a familiar inverse formula, or observing a small equation residual would not expose the erroneous scale by itself.

### 3.5 The reported scale and its contradiction

For a constant perturbation, the envelope power and logarithmic degree in the constructor are both zero. The requested observable is the source itself, so the source-observable exponent is 1 and the exponential phase power is also 1. The additional power of $v_0$ in the stored tail is therefore zero. The scale produced by the formula is

$$R_N^{\mathrm{pred}}(y) = q^{N+1} = \frac{e^{-(N+1)y}}{y^{N+1}}. \tag{7}$$

The accompanying majorant is explicitly a statement for fixed data. That hypothesis is exactly what (2) violates.[1]

**Lemma 3.1** (Elementary tail bounds)**.** *For $y > 1$ and an integer $N \geq 0$, put $m = N + 1$. Then*

$$\frac{1}{m y^m} \leq A_N(y) - \log(y - 1) \leq \frac{1}{m y^m (1 - 1/y)}. \tag{8}$$

*Proof.* The convergent real logarithm series gives

$$-\log(1 - 1/y) = \sum_{k=1}^{\infty} \frac{1}{k y^k}.$$

Subtract the first $N$ summands. The remaining summands are positive, so the first omitted one is a lower bound. For $k \geq m$, replace $1/k$ by $1/m$ and sum the resulting geometric series to obtain the upper bound. □

**Proposition 3.2** (The predicted remainder is false at every finite depth)**.** *For each fixed integer $N \geq 0$, the approximation* (6) *does not satisfy*

$$A_N(y) - \log(y-1) = O\big(R_N^{\mathrm{pred}}(y)\big) \quad \text{as } y \to +\infty.$$

*Proof.* Divide the lower bound in (8) by (7). The result is

$$\frac{A_N(y) - \log(y-1)}{R_N^{\mathrm{pred}}(y)} \geq \frac{e^{(N+1)y}}{N+1} \longrightarrow +\infty. \tag{9}$$

A Big-$O$ relation would bound this ratio by one constant for all sufficiently large $y$, which is impossible. □

This is stronger than a claim that the error constant is numerically large. No fixed error constant exists for the predicted scale. Increasing the retained depth does not cure the problem: it multiplies the exponential discrepancy in the exponent.

### 3.6 Numerical illustrations of the proved formula

Table 1 was computed with 100-decimal-digit mpmath arithmetic from the independent formulas, not from a returned package object. The exact inequality (9) is the proof; the table illustrates its size.

| $N$ | $y$ | Actual error | Predicted scale | Error/scale |
|---|---|---|---|---|
| 0 | 5 | $2.23144 \times 10^{-1}$ | $1.34759 \times 10^{-3}$ | $1.65587 \times 10^{2}$ |
| 0 | 20 | $5.12933 \times 10^{-2}$ | $1.03058 \times 10^{-10}$ | $4.97714 \times 10^{8}$ |
| 1 | 10 | $5.36052 \times 10^{-3}$ | $2.06115 \times 10^{-11}$ | $2.60074 \times 10^{8}$ |
| 3 | 10 | $2.71823 \times 10^{-5}$ | $4.24835 \times 10^{-22}$ | $6.39832 \times 10^{16}$ |
| 3 | 20 | $1.62772 \times 10^{-6}$ | $1.12803 \times 10^{-40}$ | $1.44297 \times 10^{34}$ |

Table 1: Independent evaluation of the source-predicted formulas. Full values and rigorous ratio lower bounds are in `evidence/shift-counterexample.csv`.

### 3.7 Why fixed translations must remain accepted

Let $\sigma \in \mathbb{R}$ now be a genuine fixed translation, $x = \sigma + v$. Then $a = e^{\sigma}$, $v_0 = \log y - \sigma$, and the same coefficient cancellation gives (6). The scale becomes

$$R_{N,\sigma}(y) = \left(\frac{e^{\sigma}}{y}\right)^{N+1}.$$

The error-to-scale ratio tends to

$$\frac{e^{-(N+1)\sigma}}{N+1}, \tag{10}$$

which is a finite constant. A negative constant shift such as $-3$ can make this constant large, but does not invalidate a Big-$O$ statement. The independent tests include fixed shifts $-3$, 0, and 2 precisely to distinguish this legitimate parameter dependence from the forbidden target dependence.

A blanket rule that every source-shift change must leave the literal stored scale unchanged would therefore be stronger than necessary and incompatible with the existential fixed-parameter convention. The correct requirement is that the chart data remain fixed along the declared target approach.

### 3.8 Minimal repair and diagnostic

The smallest repair is a dedicated guard after the existing variable validation and before coordinate construction:

```
If[! FreeQ[shift, y],
  fail["TargetDependentSourceShift",
    "SourceShift must be independent of the target variable; " <>
      "the remainder theorem assumes a fixed source chart.",
    <|"SourceShift" -> shift, "TargetVariable" -> y|>]];
```

The candidate generator uses the same message as a single literal string. It operates on the canonical source file, checks one exact insertion anchor, and emits a unified diff without modifying a checkout. It refuses missing, duplicate, or already-changed anchors. Its six test fixtures exercise those text-manipulation properties only; they do not establish that a patched package loads.

The guard follows the package's current structural `FreeQ` policy. It does not attempt a new scope-aware language analysis of arbitrary held binders. It also deliberately does not add `requested` to the target-free condition: a valid requested `CoreInverse` depends on $y$.

An attempted alternative—multiplying the error scale by a missing power of $a^{-1}$—would repair this one constant-perturbation example but not establish a theorem for all moving translations. Such translations can move target dependence into other coefficients or change how the nominal small coordinate compares with the original source. The current API has a fixed-chart theorem, so enforcing its hypothesis is a safer repair than silently extending its meaning.

### 3.9 Acceptance criteria and novelty

Acceptance requires an actual unpatched characterization, a patched diagnostic check, and positive controls. The supplied tests check rejection for `-Abs[y]` and `-Re[y]`, preservation of the finite coefficients for a fixed shift $-3$, and neighboring additive prototypes. Further integration controls should include automatic shift selection, fixed symbolic shifts under real parameter assumptions, and a legitimate target-dependent `CoreInverse`. These are acceptance conditions for this repair, not a new general test-framework proposal.

The distinction from the closest prior work is substantive. No stored expansion is composed with a parameter-varying inner function, and the source is not retained inside a supplied inverse. Instead, the initial source chart introduces the target into what later code treats as a constant amplitude. The minimal resolution is a refusal on an unsupported option value, not implementation of a uniform moving-parameter asymptotic theory.[5, 9]

## 4 E1: expose exact constant absorption without changing the default core

### 4.1 A grouping-sensitive exactness opportunity

The same elementary family reveals a small but useful opportunity independent of the defective shift. Compare these two calls with ordinary fixed coordinates:

```
AsymptoticExponentialCoreInverse[
  Exp[x], 1, {x,Infinity}, {y,3}]

AsymptoticExponentialCoreInverse[
  Exp[x] + 1, 0, {x,Infinity}, {y,3}]
```

Both represent the source $e^x+1$. In the first grouping, the constructor treats 1 as a nonempty perturbation and calculates a finite correction sequence. In the second grouping, the recognized core has offset 1 and an empty perturbation, so its exact inverse is available directly. These are source-predicted constructor outcomes; the identity of the underlying mathematical functions is exact.[1]

The distinction comes from `exact = rows === {}`. A nonzero constant perturbation has a row of exponent zero, hence it is not the empty case. The recognized exponential core already carries a target offset, so no new special-function inverse is necessary to absorb that constant.

More generally, whenever a core $g$ has the selected exact inverse $\phi$ and $c$ is fixed,

$$(g + c)^{-1}(y) = \phi(y - c) \tag{11}$$

on the correspondingly translated target domain. For $g(x) = e^x$, this gives (1) without constructing any of its nonzero logarithmic correction coefficients.

### 4.2 Why this should be opt-in

An exact formula is not always the representation a user requested. A user may intentionally select $e^x$ as the core in order to compare perturbations by sector depth or to retain a common core across several calculations. Silently replacing that core by $e^x + 1$ would change the meaning of the requested correction sequence even though the represented function is unchanged.

The appropriate extension is therefore explicit regrouping. A helper can absorb a perturbation known to be independent of source and target, delegate to the existing constructor with `core+c` and zero perturbation, and report that it changed the chosen core. For a mixed perturbation, a future version could split off only its constant part, but the supplied wrapper intentionally implements the pure-constant case rather than claiming that broader normalization is complete.

No exactness should be asserted if an unknown input remainder remains. The wrapper forwards constructor options, including `InputRemainder`; the underlying constructor keeps that error separate. Similarly, a user-supplied inverse of the *old* core must not be silently reused as an inverse of the regrouped core. The current wrapper leaves validation to the delegated constructor rather than forging such an identity.

### 4.3 Prototype and implementation limits

`AbsorbConstantExponentialPerturbation`, in `ReviewAdditions.wl`, is additive: it lives in `AsymptoticIncrementalReview‘` and replaces neither upstream nor `System` definitions. It checks that the absorbed term is independent of both variables and that the target constructor has definitions before delegating. The constructor remains responsible for exact real input, branch, depth, and remainder-option validation.

The wrapper deliberately accepts immediate option rules only. It does not pretend to resolve the repository's broader delayed/nested option-policy questions, which are already part of prior review work. This limitation is explicit in the fallback diagnostic and the README. The WL implementation is unexecuted; the independent Python tests validate the exact translation identity, not the wrapper's evaluation semantics.

The expected work reduction is structural: the pure-constant regrouped case takes the existing empty-perturbation branch instead of computing all requested coefficient expressions. No elapsed-time improvement in the package was measured. This is also not a recurrence-termination optimization: for the unabsorbed logarithm example, every coefficient $-1/n$ is nonzero. The exactness comes from choosing a different core, not from detecting zeros sooner.

# 5 E2: rational moment arithmetic for the existing Lerch adapter

## 5.1 A concrete replacement boundary

The existing Lerch path expands in the reciprocal of a large positive argument while keeping the other parameters fixed. The defining sum is

$$\Phi(z, s, a) = \sum_{n=0}^{\infty} \frac{z^n}{(a+n)^s}, \qquad |z| < 1, \quad a > 0, \tag{12}$$

which is absolutely convergent for fixed real $s$ in this domain. The defining sum and its relation to polylogarithms are standard; the proposal below is a specific computational specialization, not a claim of a new identity.[19, 20]

The inspected source evaluates moments by `PolyLog[-k,z]` for positive integer $k$, gives the zeroth moment separately, and places each coefficient behind a bounded `FullSimplify`. The tail constant requests moments again at `Abs[z]`. This is a reasonable general exact-expression route, but exact rational parameters admit a smaller arithmetic kernel with no such special-function or simplification calls.[3]

The proposed first integration boundary is intentionally narrow:

$$z, s \in \mathbb{Q}, \qquad |z| < 1,$$

with bounded moment degree. Other admissible exact parameters continue to use the existing path. This proposal does not change what counts as a nonzero block, the absolute exponent convention, the target approach, or the theorem justifying the tail.

## 5.2 Derivation of the moment recurrence

Define

$$M_0(z) = \frac{1}{1-z}, \qquad M_k(z) = \sum_{n=0}^{\infty} n^k z^n \quad (k \geq 1). \tag{13}$$

The $n = 0$ contribution to $M_0$ is included; this bookkeeping convention must not be lost by calling every $M_k$ a polylogarithm. For $k \geq 1$, $M_k = \mathrm{Li}_{-k}(z)$, whereas $M_0 = 1 + \mathrm{Li}_0(z)$.

Absolute convergence on compact subsets of $|z| < 1$ permits termwise differentiation. Hence

$$M_{k+1}(z) = z \frac{d}{dz} M_k(z).$$

Write

$$M_k(z) = \frac{P_k(z)}{(1-z)^{k+1}}, \qquad P_0(z) = 1. \tag{14}$$

Differentiation gives the polynomial recurrence

$$P_{k+1}(z) = z(1-z)P_k'(z) + (k+1)zP_k(z). \tag{15}$$

All coefficients remain integers. For $P_k(z) = \sum_j p_{k,j} z^j$, an implementation can initialize a zero coefficient array and accumulate

$$p_{k+1,j} \mathrel{+}= j p_{k,j}, \qquad p_{k+1,j+1} \mathrel{+}= (k+1-j) p_{k,j}. \tag{16}$$

It then evaluates $P_k(z)$ by Horner's rule and updates the denominator by one multiplication by $1 - z$.

The first values provide transparent boundary checks:

$$M_0 = \frac{1}{1-z}, \quad M_1 = \frac{z}{(1-z)^2}, \quad M_2 = \frac{z(1+z)}{(1-z)^3}.$$

At $z = 1/2$ the first six moments are

$$2,\ 2,\ 6,\ 26,\ 150,\ 1082.$$

At $z = 0$, they are $1, 0, 0, \ldots$. The supplied implementations cover both cases without evaluating a special function.

### 5.3 Coefficient generation and exact zeros

Expanding the elementary factor $(1 + n/a)^{-s}$ at zero gives the formal coefficient of $a^{-s-k}$:

$$c_k(z, s) = \frac{(-1)^k (s)_k}{k!} M_k(z). \tag{17}$$

For rational $z$ and $s$, these coefficients are rational. The scalar factor can be updated without repeated factorial or Pochhammer evaluation:

$$t_0 = 1, \qquad t_{k+1} = -t_k \frac{s+k}{k+1}, \qquad c_k = t_k M_k. \tag{18}$$

The table retains exact zeros. This is essential because the repository's term goal counts nonzero displayed blocks, while the remainder theorem is indexed by Taylor degree. A consumer must not confuse "the fifth nonzero coefficient" with Taylor degree five. For a nonpositive integer $s = -m$, the scalar factor vanishes after $k = m$ and the source expansion is finite; for $z = 0$, all positive-degree moments vanish. At $z = 1/2$, $s = -2$, the coefficient table starts

$$(2, 4, 6, 0, 0, \ldots),$$

corresponding to the exact polynomial $2a^2 + 4a + 6$.

For negative rational $z$, coefficients may have cancellations. A zero entry is not a reason to relabel all subsequent indices, and one isolated zero is not by itself an exact termination certificate. The proposed table changes arithmetic representation only; the caller keeps its existing stopping and frontier logic.[3]

### 5.4 Preserving the existing tail theorem

The useful optimization must preserve the original bound, not merely reproduce coefficients. With $N$ denoting the first omitted Taylor degree, the repository uses

$$D = \max\{0, \lceil -s - N \rceil\}, \tag{19}$$

$$B_N(z, s) = \frac{|(s)_N|}{N!} \sum_{j=0}^{D} \binom{D}{j} M_{N+j}(|z|), \tag{20}$$

and, for $a \geq 1$,

$$\left|\Phi(z,s,a) - \sum_{k=0}^{N-1} c_k(z,s)a^{-s-k}\right| \leq B_N(z,s)a^{-s-N}. \tag{21}$$

This is the existing source theorem, restated here to specify what the arithmetic replacement must preserve.[3]

For completeness, its elementary mechanism can be checked directly. Taylor's theorem bounds the remainder of $h(t) = (1+t)^{-s}$ on $t \geq 0$ by

$$\frac{|(s)_N|}{N!}t^N(1+t)^D.$$

Put $t = n/a$ and use $a \geq 1$ to replace $(1+n/a)^D$ by $(1+n)^D$. Sum against $|z|^n$, expand the integer power $(1+n)^D$, and obtain (20). The zeroth-moment convention is required when a degree zero occurs. A rational moment table computes exactly the same constant.

One local table suffices when $z \geq 0$. For $z < 0$, use a table at $z$ for coefficients and one at $|z|$ for the absolute bound. The source theorem is fixed-parameter and does not become uniform as $z$ approaches 1. The optimized arithmetic must not introduce that stronger claim.

### 5.5 Independent oracle and complexity

An independent exact oracle follows from falling factorials:

$$M_k(z) = \sum_{j=0}^{k} \left\{ {k \atop j} \right\} j! \frac{z^j}{(1-z)^{j+1}}. \tag{22}$$

Here the braces are Stirling numbers of the second kind. The identity follows by substituting the falling-factorial expansion of $n^k$ into the geometric sum. The supplied oracle builds Stirling rows, while the production prototype builds polynomial numerator arrays by (16); this separates the two recurrence mechanisms.

Producing all moments through degree $K$ takes $O(K^2)$ arithmetic operations with coefficient-array updates and Horner evaluation. This is *not* a quadratic bit-complexity assertion. The integer polynomial coefficients and rational values grow, especially when the rational input has a large numerator or denominator. Storing the current numerator costs $O(K)$ integer entries; retaining the whole output costs $K+1$ rational entries, whose bit lengths are not constant.

The prototype degree cap is 256. It is a simple operational limit, not a proof of a uniform memory bound for arbitrarily large rational input. There is no global memoization, no retained symbolic parameter state, and no claim that a Python timing predicts the corresponding evaluator's cost. An integrated backend can keep the package's existing resource checks around the rational arithmetic result.

### 5.6 Executed prototype measurements

The independent implementation uses Python's `fractions.Fraction`. Table 2 records the best of three local runs at $z = 1/2$. It measures the supplied algorithm only; no upstream baseline was available, so a speedup ratio would be unsupported.

The benefit presently established is algorithmic dependency reduction: rational inputs can be handled by integer and rational arithmetic without a per-moment special-function call. The practical value for Mathics is plausible for that reason, but actual compatibility and performance remain to be measured. The WL implementation is an additive prototype, not an installed adapter, and its success cannot be inferred from the equivalent Python algorithm.

| Highest degree $K$ | Moments returned | Best elapsed time | Last numerator bits |
|---:|---:|---:|---:|
| 16 | 17 | 0.241 ms | 54 |
| 32 | 33 | 0.848 ms | 136 |
| 64 | 65 | 3.281 ms | 331 |
| 128 | 129 | 13.246 ms | 785 |

Table 2: Python-only prototype timings from `evidence/prototype-timing.json`; machine-dependent observations, not Wolfram or Mathics performance results.

## 6 Executed checks, supplied tests, and what they do not prove

### 6.1 Independent Python population

The recorded run used Python 3.13.5 and mpmath 1.3.0, with 100 decimal digits for numerical observations. The suite contains 32 unittest methods and completed with no failures, errors, or skips. The raw transcript and machine-readable scope are included. Exact rational calculations are performed by `Fraction`, not by converting machine approximations to apparent exact data.

| Group | Methods | Main obligations checked |
|---|---:|---|
| Source-shift models | 9 | Positive tail inequalities; exponential ratio discrepancy; eventual-domain samples; fixed-shift controls; coefficient recurrence; amplitude cancellation; constant translation identity; invalid model arguments. |
| Rational moments | 17 | Known values; zero parameter; independent Stirling oracle; first two identities; signed/absolute controls; termination; Pochhammer factors; four numerical Lerch bound examples; domain and degree refusals. |
| Patch fixtures | 6 | Unique insertion; source preservation away from insertion; missing and ambiguous anchors; already-patched refusal; non-mutating diff generation. |
| Total | 32 | Independent models and prototypes only. |

The moment-oracle method compares degrees 0 through 24 at six rational values of $z$, including zero and negative values. Those internal loops remain part of one method in the reported count. The numerical Lerch checks test selected values against (21); they are illustrations of a preserved bound, not a numerical proof of the general theorem.

The exact proof of F1 is stronger than the numerical assertions: the lower bound holds for every $y > 1$ and every fixed finite depth. Conversely, no number of independent model tests establishes the public constructor's evaluation path. That is why the evidence JSON identifies the missing kernel execution rather than recording a fictitious package pass count.

### 6.2 Eight desired-contract tests, unexecuted

`ReviewRegressions.wlt` contains two tests for the new target-dependent-shift diagnostic, a fixed-shift positive control, an exact constant-absorption case, three rational-moment/coefficient cases, and a refusal of an inexact rational-prototype argument. They are written as desired contracts for a patched package plus the additive helper file.

The independent characterization script serves a different purpose: load the unpatched package, record

the kernel version and the returned head, and inspect the expression, scale, target-domain value, and numerical error ratio for depths 0, 1, and 3. Its output must be retained even if a request refuses or stays unevaluated. It is not an acceptance suite with a predetermined pass total.

A fresh patched kernel and a fresh unpatched kernel should be separate sessions. In particular, loading the additive helper does not install the F1 fix: the guarded source change still needs to be applied to the actual checkout. The supplied patch generator only emits a candidate diff.

### 6.3 Lexical and artifact checks

The three supplied WL/WLT files passed a limited scanner for paired delimiters, quoted strings, and nested comments. That scanner is included so the scope is inspectable. It is not a Wolfram parser, does not validate pattern precedence, does not evaluate definitions, and cannot establish Mathics semantics.

The article is supplied as a self-contained LaTeX file and a rendered PDF. The build and visual-inspection records describe the artifact checks separately from software tests. No checksum files are included, and no claim is based on a checksum generated by this review.

## 7 Prioritized implementation decisions

### 7.1 First: close the fixed-chart hole

F1 should be handled before either optimization. The change is small, the theorem violation is severe, and the positive compatibility boundary is clear: preserve fixed translations and ordinary target-dependent inverse expressions while refusing target-dependent chart data. The characterization should establish the current public behavior in Wolfram 15 and in the actual Mathics configuration used by the project, without assuming that an earlier proof refusal is an identical reproduction of the native symptom.

The diagnostic should explain the fixed-chart restriction rather than presenting the expression as a malformed exponential core. That makes the required user action clear: choose a fixed source translation, or reformulate a genuinely moving-coordinate problem outside this constructor's stated theorem.

### 7.2 Second: decide whether exact core regrouping belongs in the API

E1 is best treated as an explicit representation choice. The pure-constant helper already demonstrates the minimal change in mathematical input grouping. An integrated version can retain the original core and perturbation alongside the regrouped pair, but must not silently promise the original correction sequence after changing its base.

The deciding tests are not just coefficient equality. They are preservation of the original source function, translation of the target domain, correct handling of a remaining input remainder, and rejection or revalidation of a supplied old-core inverse. Success would establish a small new exactness capability rather than a broad rewrite of inverse dispatch.

### 7.3 Third: integrate rational moments only behind a narrow gate

E2 should begin with rational $z$ and $s$, exact values, and a bounded requested degree. It must preserve the original Taylor index for a frontier, count nonzero blocks according to the existing convention, and return the same tail constant and hypotheses as the general implementation. Fall back outside that gate instead of expanding the implementation claim to all exact parameters.

An actual comparison should distinguish coefficient generation from bound construction, use the same requested and achieved order, and compare the rational and existing paths in both runtimes. The present report supplies the exact oracle and Python observations needed to design that comparison; it supplies no invented kernel timing or throughput target.

### 7.4 What is intentionally not proposed again

This report does not add another general registry redesign, uniform-asymptotics program, capability taxonomy, full-suite policy, persistence format, broad caching plan, or list of missing special functions. Those are already represented in the exclusion baseline. Its forward-looking work is restricted to enforcing the fixed-chart hypothesis, exposing optional constant absorption, and replacing one rational computational dependency in an existing adapter.[5, 6, 7, 8]

## 8 Conclusion

The principal incremental result is a concrete separation between correct coefficients and an incorrect error contract. A moving source translation can be algebraically canceled out of every retained coefficient while remaining hidden in the claimed remainder. For $e^x + 1$, the contradiction is elementary, applies at every finite depth, and is resolved by enforcing the fixed-chart condition already assumed by the constructor.

The two accompanying opportunities are deliberately smaller than a new transseries engine. Constant absorption exploits an exact inverse the chosen grouping hides. Rational moment tables exploit a finite arithmetic identity inside a special-function adapter already present. Both can be developed without weakening the branch and remainder boundaries on which the package depends.

The evidence supports a high-priority source fix and two concrete implementation experiments. It does not support a claim that a patched package has passed Wolfram 15 or Mathics3 tests. The archive preserves that distinction with proofs, executable independent checks, unexecuted integration specifications, source references, and an explicit novelty ledger.

## A Reproduction and artifact inventory

### A.1 Independent calculations

From the archive root, with Python and the listed dependency installed:

```
python -m pip install -r code/requirements.txt
python code/run_evidence.py
python code/check_wl_lexical.py
```

The evidence generator rewrites its own result files. The exact moment module itself uses the standard library; mpmath is needed by the numerical test population and sample generator. Timing observations will vary by environment.

### A.2 Candidate patch and native tests

Given an actual checkout of the pinned repository:

```
python code/prepare_shift_patch.py /path/to/Asymptotic > shift.patch
```

Inspect and apply that diff separately. The generator does not perform either action. Regenerate the standalone using the project's existing builder after a canonical source change.[4] In a fresh Wolfram kernel, the intended test workflow is:

```
Get["/path/to/Asymptotic/src/Kernel/AsymptoticAnalysis.wl"];
Get["/path/to/review/code/ReviewAdditions.wl"];
TestReport["/path/to/review/code/ReviewRegressions.wlt"]
```

These are instructions, not a transcript of an executed session. Mathics requires a separate actual run. Its support for the MUnit wrapper is not assumed; the scalar test expressions can be adapted to the repository's existing portable runner.

### A.3 File roles

| File | Role |
|---|---|
| `article/article.tex`, `article/article.pdf` | Editable article and compiled, visually inspected output. |
| `code/review_math.py` | Exact rational algorithms, logarithm-tail bounds, and independent core models. |
| `code/test_review.py` | The 32-method independent suite. |
| `code/run_evidence.py` | Reproduces tests, samples, and Python-only timing observations. |
| `code/prepare_shift_patch.py` | Non-mutating, exact-anchor candidate diff generator. |
| `code/ReviewAdditions.wl` | Unexecuted additive WL prototypes for E1 and E2. |
| `code/ReviewRegressions.wlt` | Eight unexecuted desired-contract tests. |
| `code/characterize_shift.wl` | Unexecuted baseline observation script. |
| `code/check_wl_lexical.py` | Limited lexical scanner, not an evaluator. |
| `evidence/independent-results.json` | Actual independent count and execution boundaries. |
| `evidence/independent-test-output.txt` | Raw unittest transcript. |
| `evidence/shift-counterexample.*` | CSV/JSON evaluations of the independently derived witness formulas. |
| `evidence/prototype-timing.json` | Python-only rational moment timings. |
| `evidence/findings.json`, `evidence/novelty-ledger.csv` | Claim-specific classification and closest prior-work distinctions. |
| `evidence/source-map.json`, `evidence/scope.json` | Inspected source ranges and explicit limits of the review. |
| `evidence/wl-lexical-check.json` | Delimiter/comment/string check results. |
| `evidence/pdf-build.json` | PDF build and rendered inspection record. |

## B Source anchors and bibliography

All repository references below are pinned to `8cee870`. The full commit is printed on the title page and recorded in the machine-readable evidence. A source link describes the snapshot inspected; it does not imply that the same text remains on a later `main` branch.

The primary F1 anchors are the source-coordinate construction, the exact core inverse, the guard, coefficients, and error scale, and the exponential amplitude parser. E2 is localized by the named helpers `dirichletSpecialMoment`, `dirichletLerchCoefficient`, `dirichletLerchBoundConstant`, and `dirichletLerchForward`. The separate source map lists broader inspected files without presenting them as entirely verified modules.

## References


[1] Vladimir Reshetnikov, Asymptotic repository, `src/Kernel/ExponentialCorePerturbation.wl`. Source-coordinate admission, exact cores, sector coefficients, and fixed-data majorant.

[2] Asymptotic repository, `src/Kernel/LambertInverse.wl`. In particular, `lambertExponentialTerm` and `lambertExponentialCore`.

[3] Asymptotic repository, `src/Kernel/DirichletSpecialFunctions.wl`. Existing Zeta and Lerch constructors, moments, coefficients, and tail constants.

[4] Asymptotic repository, `src/Kernel/README.md`. Modular source map, Mathics architecture, validation scope, and standalone generation.

[5] Asymptotic repository, `docs/development/CODE_REVIEW_STATUS.md`. Maintained finding and proposal register, including complete early-wave crosswalks.

[6] Asymptotic repository, `docs/development/WAVE_3_INTAKE.md`. Wave-3 intake and crosswalk.

[7] Asymptotic repository, `docs/development/WAVE_4_INTAKE.md`. Wave-4 intake and crosswalk.

[8] Asymptotic repository, `external-reports/code-review/wave-5/README.md`. Retained wave-5 reports, overlaps, retired copies, and evidence limitations.

[9] Asymptotic repository, `external-reports/code-review/wave-5/code-review-43/README.md`. Closest source-symbol validation finding and its stated scope.

[10] Asymptotic repository, `src/Kernel/MathicsAssumptions.wl`. Realness and sign proof helpers, especially `mathicsRealProof`.

[11] Asymptotic repository, `src/Kernel/MathicsCoreFunctions.wl`. Construction-site ProductLog and timeout handling.

[12] Asymptotic repository, `src/Kernel/MathicsTaylor.wl`. Bounded defining-series fallback.

[13] Asymptotic repository, `src/Kernel/NativeSpecialFunctions.wl`. Analytic native-series import and truncation.

[14] Asymptotic repository, `src/Kernel/GammaInverseChecks.wl`. Independent finite-model normalized residuals and numerical original-equation checks.

[15] Asymptotic repository, `src/Kernel/BarnesInverseChecks.wl`. Barnes residual normalization.

[16] Asymptotic repository, `src/Kernel/BarnesForward.wl`. Exact argument shifts and finite Barnes model.

[17] Asymptotic repository, `src/Kernel/LogarithmicScales.wl`. Finite logarithmic hierarchy and coefficient Euler derivative.

[18] Asymptotic repository, `src/Kernel/SeriesEnvelopeArithmetic.wl`. Composite error arithmetic and observable transport conditions.

[19] NIST Digital Library of Mathematical Functions, §25.14(i), equation 25.14.1, Lerch's transcendent. `https://dlmf.nist.gov/25.14.E1`. Accessed 10 September 2026. Used for the defining sum and domain, not as a source of the new code-review finding.

[20] NIST Digital Library of Mathematical Functions, §25.12(ii), equation 25.12.10, polylogarithm defining series. `https://dlmf.nist.gov/25.12.E10`. Accessed 10 September 2026.

[21] Wolfram Research, *Abs: Absolute Value (Modulus)*, Wolfram Language documentation. `https://reference.wolfram.com/language/ref/Abs.html`. Accessed 10 September 2026. This is semantic documentation, not a transcript of a Wolfram 15 evaluation.